# Tema 2.- Transformers e mecanismo de atención

Marcos Fernández Pichel

4º GrIA - Universidade de Santiago de Compostela

30 de setembro do 2026

# Que veremos neste tema?

- Mecanismos de atención.
- Arquitectura Transformer
  - Tokenización.
  - Embeddings Lookup Table.
  - Positional Encodings.
  - Encoder Stack.
  - Decoder Stack.
- Modelos seq2seq.
- Modelos encoder only.
- Modelos decoder only.
- Conceptos avanzados de Transformers.
- Conclusións do tema.

# Introdución

## IA da linguaxe

Nos últimos anos, conseguíronse modelos capaces de capturar a **semántica** das palabras e de realizar representacións **dependentes do contexto**. Porén:

- Modelos como ELMo procesan o texto palabra a palabra. Isto non se pode **paralelizar** facilmente, polo que resulta lento para secuencias longas.
- As palabras **afastadas** seguen tendo **pouca influencia**, xa que a información ten que viaxar a través de moitos pasos da LSTM, a pesar de que se xestione mediante a memoria.
- ELMo **non pondera dinamicamente** que partes do contexto son máis importantes para cada palabra.

# Mecanismos de atención

## Definición

A atención permite que, ao procesar unha palabra, o modelo mire activamente todas as demais palabras da frase e decida cales son relevantes para comprendela.

## IMPORTANTE

*Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. Advances in neural information processing systems, 30.*

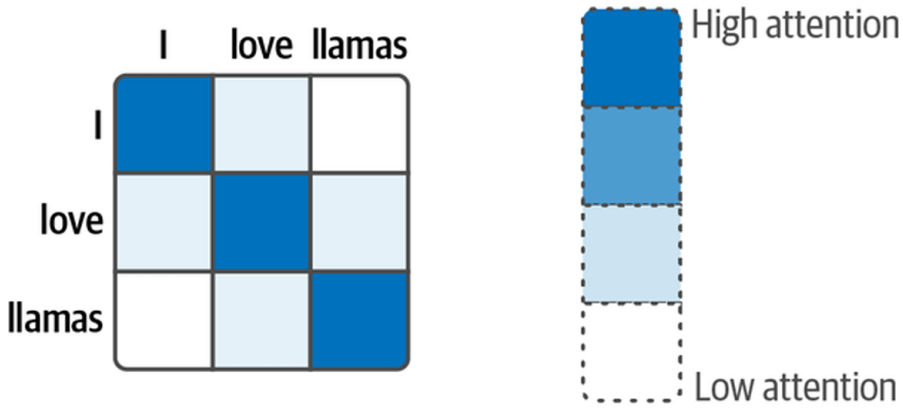



Figura: Figura extraída do libro "Hands on Large Language Models".

# Mecanismos de atención

## Exercicio

Se tivésemos a frase “The animal didn’t cross the street because **it** was too tired”, cal dos outros tokens é máis importante para o token “it”? Que outro problema observades nesta secuencia de entrada?

# Mecanismos de atención

## Exercicio

Se tivésemos a frase "The animal didn't cross the street because **it** was too tired", cal dos outros tokens é máis importante para o token "it"? Que outro problema observades nesta secuencia de entrada?

## Exercicio

E se agora a secuencia fose "The book is on the table because **it** is heavy"?

# Arquitectura Transformer

## Arquitectura Transformer

- A arquitectura dos Transformers implementa a atención e permite superar as limitacións do cómputo secuencial das LSTM.
- Os Transformers procesan todos os tokens en paralelo empregando atención, superando a dependencia secuencial das LSTM e permitindo unha paralelización eficiente.
- Consiste nunha serie de bloques de **encoders** e **decoders** apilados uns enriba dos outros.
- Como veremos máis adiante, trátase dunha arquitectura **autoregresiva**, é dicir, necesita consumir cada palabra xerada antes de xerar a seguinte.

# Arquitectura Transformer

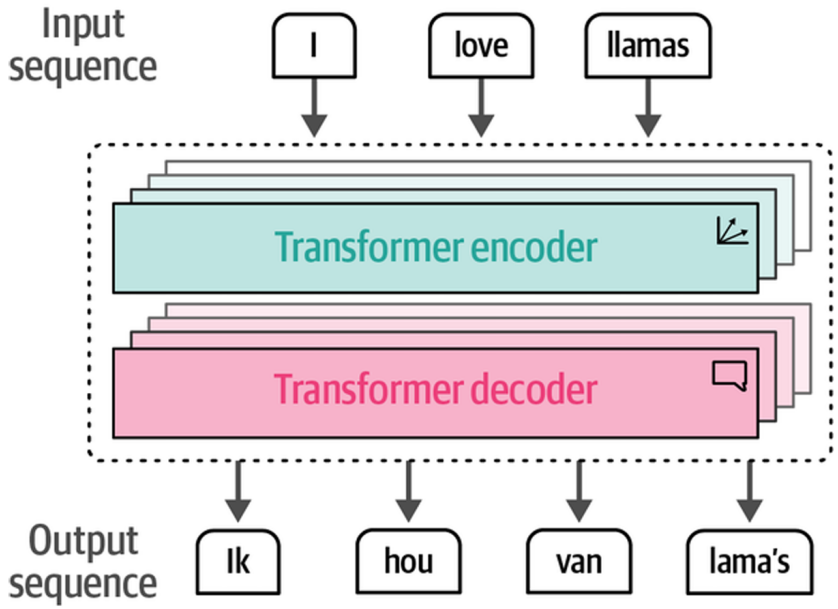



Figura: Transformer completo para tradución automática entre idiomas. Figura extraída do libro "Hands on Large Language Models".

Imos ver que os Transformers poden ser **encoder-only** (modelo de BERT), **decoder-only** (GPT) ou **encoder-decoder** (T5).

# Arquitectura Transformer

## Arquitectura Transformer

Cal credes que é o primeiro paso cando alimentamos un Transformer cunha secuencia de texto?

# Tokenización

## Tokenización

- Igual que con modelos previos, o primeiro paso na arquitectura Transformer é a **tokenización**.
- En Word2Vec partíamos dun vocabulario de **palabras completas**. Cada palabra tiña o seu vector único.
- Para mellorar a capacidade de captar **relacións morfolóxicas** entre palabras, chegaron as técnicas baseadas en **subpalabras** como fastText.
- Consolidouse con **WordPiece** e **BPE (Byte Pair Encoding)** nos Transformers.

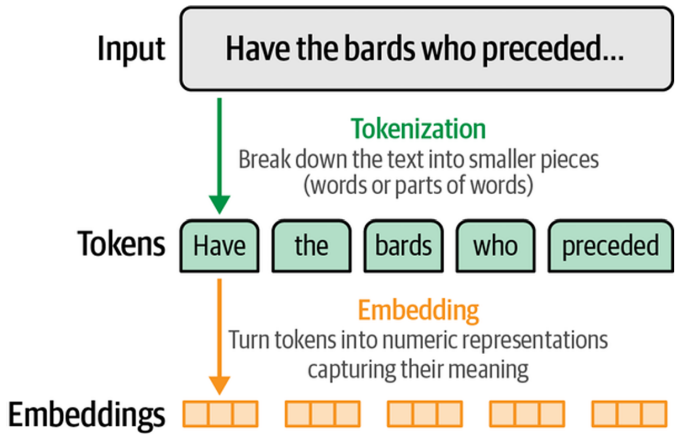



Figura: Figura extraída do libro "Hands on Large Language Models".

# Tokenización

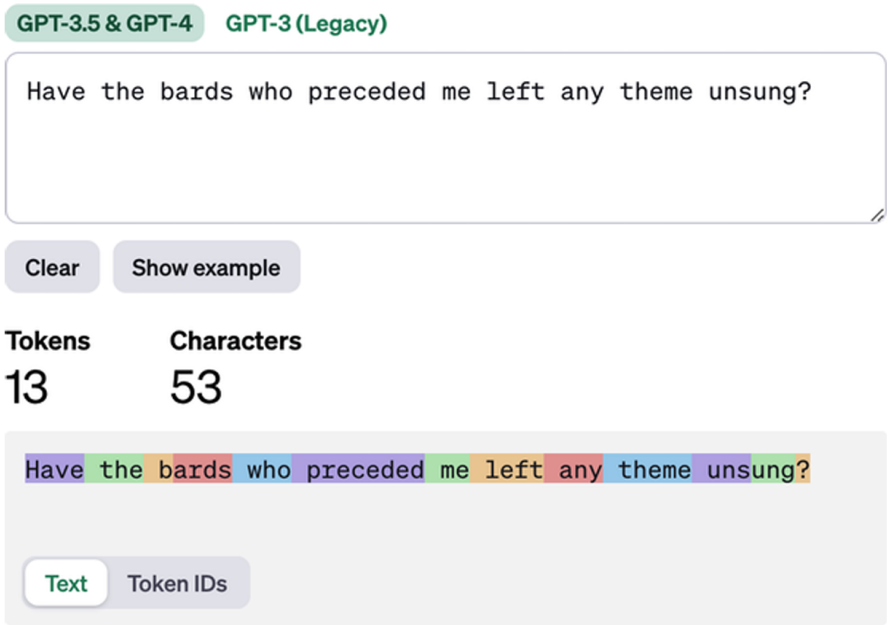



Figura: Figura extraída do libro "Hands on Large Language Models".
`https://platform.openai.com/tokenizer`

## Proba

Probádeo vós mesmos/as e comprobade que non é o mesmo **token** que palabra e que ten soporte **multilingüe**.

# Tokenización

## Tokenización

Probade nesta outra páxina agora, probade a mesma frase (ex. “aprendizaxe profundo”): `https://huggingface.co/spaces/Xenova/the-tokenizer-playground`
Probade bert-base, T5 e GPT-4.

# Tokenización

## Como divide un tokenizador o texto?

- A tokenización é **dependente do modelo**. BPE (modelos de GPT) e WordPiece (modelo de BERT).
- O tokenizador **adestrase** nun dataset para aprender cal é o mellor vocabulario que o representa.
- Xera unha correspondencia entre tokens e **IDs** tanto na entrada como na saída do modelo.

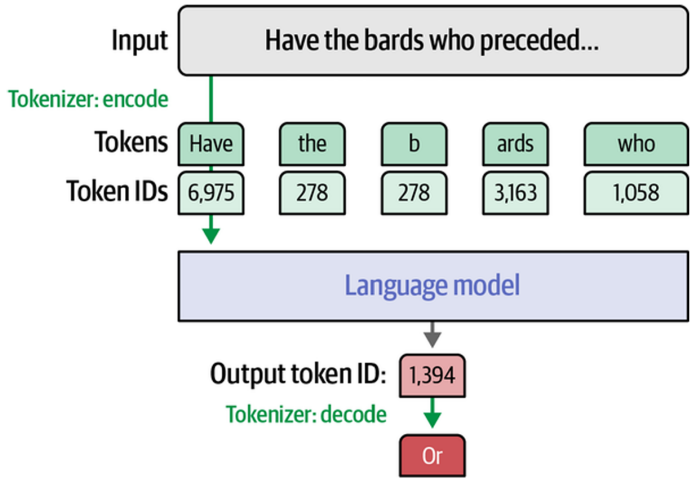



Figura: Figura extraída do libro "Hands on Large Language Models".

# Tokenización

## Byte Pair Encoding (BPE)

- No seu adestramento, parte de tokens de **caracteres individuais** e logo combina os pares máis frecuentes para formar tokens máis longos.
- Exemplo de tokenización: "lowest" $\rightarrow$ "low", "est".
- Úsase nos modelos de GPT.

## WordPiece Tokenizer

- No seu adestramento, parte de tokens de **caracteres individuais** e logo constrúe un vocabulario de subpalabras (*subtokens*) baseándose en maximizar unha probabilidade.
- Exemplo de tokenización: Se tes a palabra "unaffable", WordPiece pode tokenizala en "un", "##aff", "##able". O prefixo "##" indica que o token é unha continuación da palabra anterior.
- Úsase no modelo de BERT.

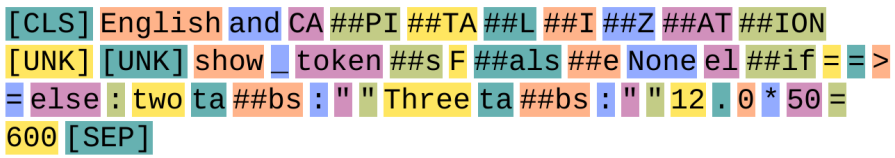
# Tokenización

## Tokens especiais

- Son símbolos engadidos ao texto durante a tokenización que **non** proveñen **directamente do texto orixinal**, pero teñen un propósito **estrutural** ou funcional para o modelo.
- Úsanse, por exemplo, para marcar inicios ou finais de secuencias ([CLS], [SEP], <s></s> en BPE, etc.)
- Modelos como BERT usan WordPiece co token CLS para representar a secuencia enteira de entrada. Verémolo máis adiante.

## Hugging Face

`https://huggingface.co/docs/tokenizers/index`



# Arquitectura Transformer

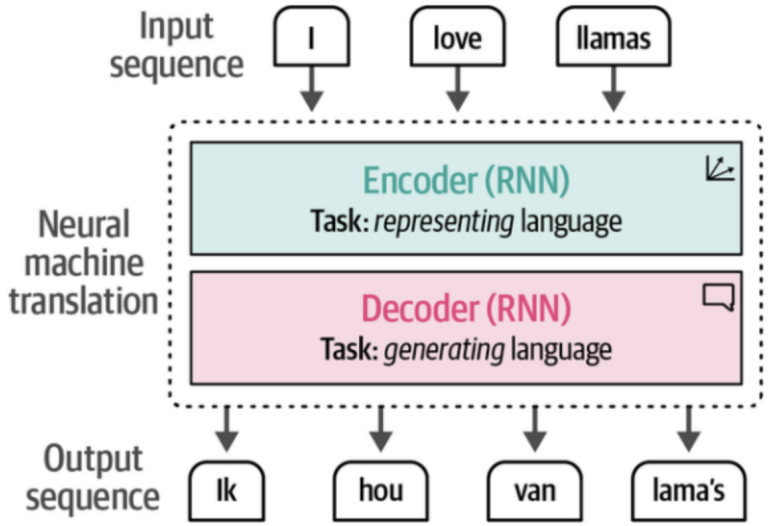



Figura: Figura extraída do libro "Hands on Large Language Models".

## Entrada ao Transformer

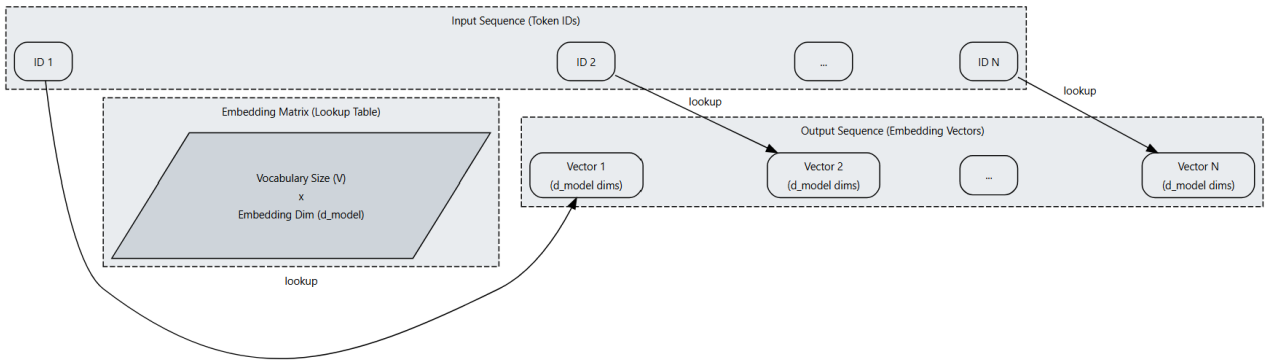
Antes de explicar a estrutura do Transformer, temos que entender como entran os datos no Transformer.

Xa vimos o primeiro paso, a tokenización (tokens a IDs).

# Embeddings lookup table

## Embeddings lookup table

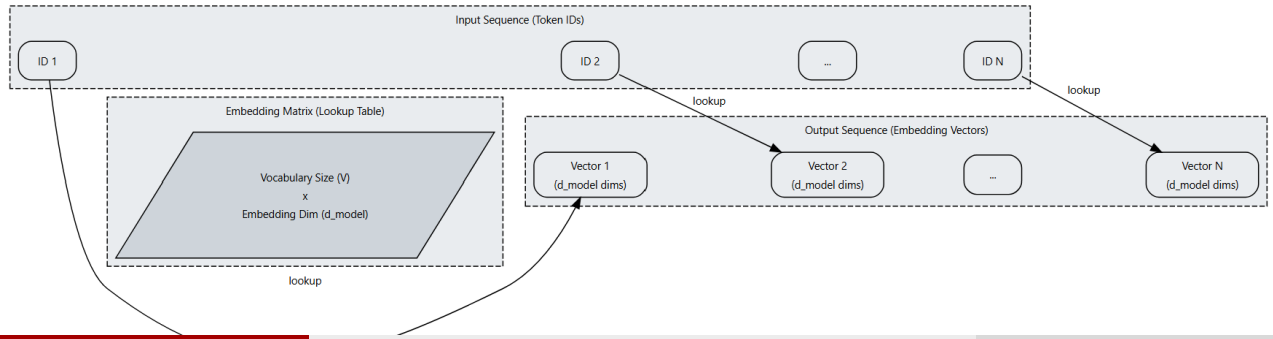
- Antes de entrar no encoder do Transformer, necesitamos transformar os **token IDs** nalgo comprensible para unha rede: vectores numéricos ou **embeddings**.
- Para iso, necesitamos unha **lookup table**, de tamaño $V \times d_{\text{model}}$, onde $V$ é o tamaño do vocabulario e $d_{\text{model}}$ a dimensionalidade dos vectores de embeddings; no artigo orixinal $d_{\text{model}} = 512$.
- Cada token ID ten un embedding asociado na *lookup table*.



# Embeddings lookup table

## Embeddings lookup table

Estes vectores de embeddings non son simplemente representacións arbitrarias. Son parámetros **adestrables**.
Inicialmente poden ter valores aleatorios, pero durante o proceso de adestramento do modelo, estes vectores axústanse mediante backpropagation. O obxectivo é que os embeddings aprendidos capturen similitudes semánticas entre os tokens.
Estes embeddings son **estáticos**.
Unha vez que a entrada accede ao modelo, eses embeddings iniciais vanse transformando en cada capa do encoder.



# Embeddings lookup table

Exemplo do visto ata o de agora. Imaxinade que temos a frase: “The street was unwalkable”.

- Tokenización con WordPiece: [“The”, “street”, “was”, “un”, “##walk”, “##able”, “.”]
- Conversión a token IDs: [101, 202, 303, 404, 505, 606, 707]
-

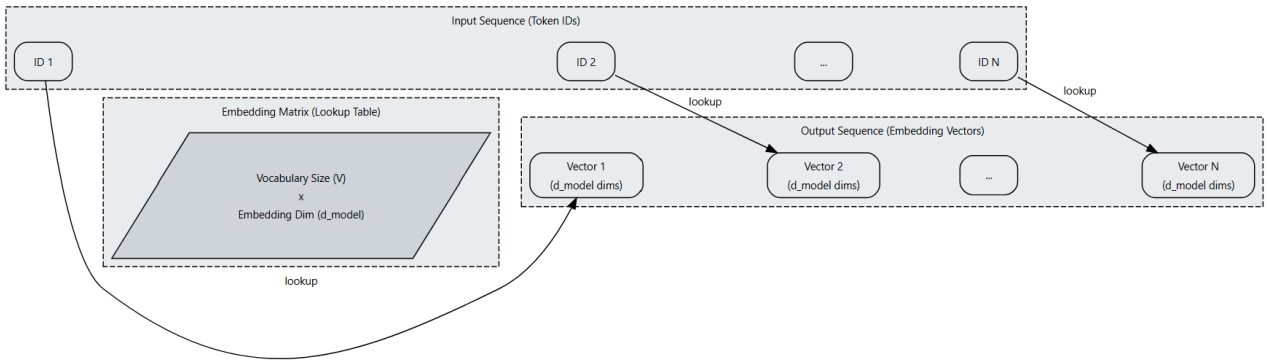
| Token | ID | Embedding en $\mathbb{R}^4$ |
|---|---|---|
| The | 101 | $[0{,}25, -0{,}10, 0{,}33, 0{,}80]$ |
| street | 202 | $[0{,}40, 0{,}05, -0{,}20, 0{,}10]$ |
| was | 303 | $[0{,}15, -0{,}30, 0{,}50, -0{,}25]$ |
| un | 404 | $[0{,}60, -0{,}15, 0{,}45, 0{,}00]$ |
| ##walk | 505 | $[0{,}10, 0{,}55, -0{,}05, 0{,}35]$ |
| ##able | 606 | $[0{,}20, 0{,}10, 0{,}60, -0{,}40]$ |
| . | 707 | $[0{,}00, 0{,}00, 0{,}00, 0{,}00]$ |

- Que sucede co embedding da palabra “play”?

# Embeddings lookup table

## Embeddings lookup table

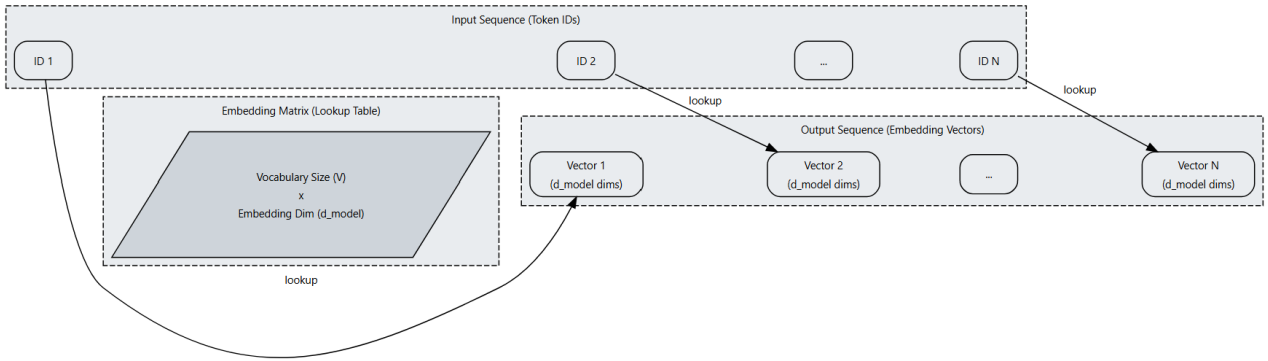
Pero entón, se o vocabulario é fixo, por que se adapta mellor ca Word2Vec ao novo vocabulario? Pensade agora no embedding da palabra “walkable”.



# Embeddings lookup table

## Embeddings lookup table

Pero entón, se o vocabulario é fixo, por que se adapta mellor ca Word2Vec ao novo vocabulario? SUBTOKENS!



# Positional Embeddings

## Positional Embeddings

- Outra das vantaxes dos Transformers é que procesan toda a **entrada en paralelo**.
- Para o Transformer, a secuencia de entrada é como unha bolsa de tokens.
- Imaxinade o conxunto de tokens {Dog, man, bites, the}.
- Como pode saber o modelo a orde da secuencia?

# Positional Embeddings

## Positional Embeddings

- Ao pasar de redes recorrentes a Transformers, atopamos unha diferenza fundamental na forma de procesar os datos: as redes recorrentes procesan os tokens segundo a orde de entrada.
- Porén, os Transformers poden calcular a atención de maneira **simultánea** entre os **tokens de entrada**.
- Imaxinade as frases: "Dog bites man" e "Man bites dog".
- Para o cálculo da atención do Transformer, trataríase da mesma colección de vectores de entrada, pero simplemente ordenados de maneira diferente.
- Polo tanto, é necesario proporcionar algún tipo de información sobre a **posición** na **secuencia de entrada**.

# Positional Embeddings

## Positional Embeddings

- Idea: crear un vector único para cada posición na secuencia e sumar ese vector co embedding do token.
- $final_{\text{embedding}}(pos) = input_{\text{embedding}}(pos) + PE(pos)$
- Existen diferentes implementacións dos PE, pero a que se adoita empregar é a baseada en **senos** e **cosenos** de **diferentes frecuencias**.
- A función seno aplícase ás dimensións pares do vector de positional embedding (PE), e a función coseno ás dimensións impares.

# Positional Embeddings

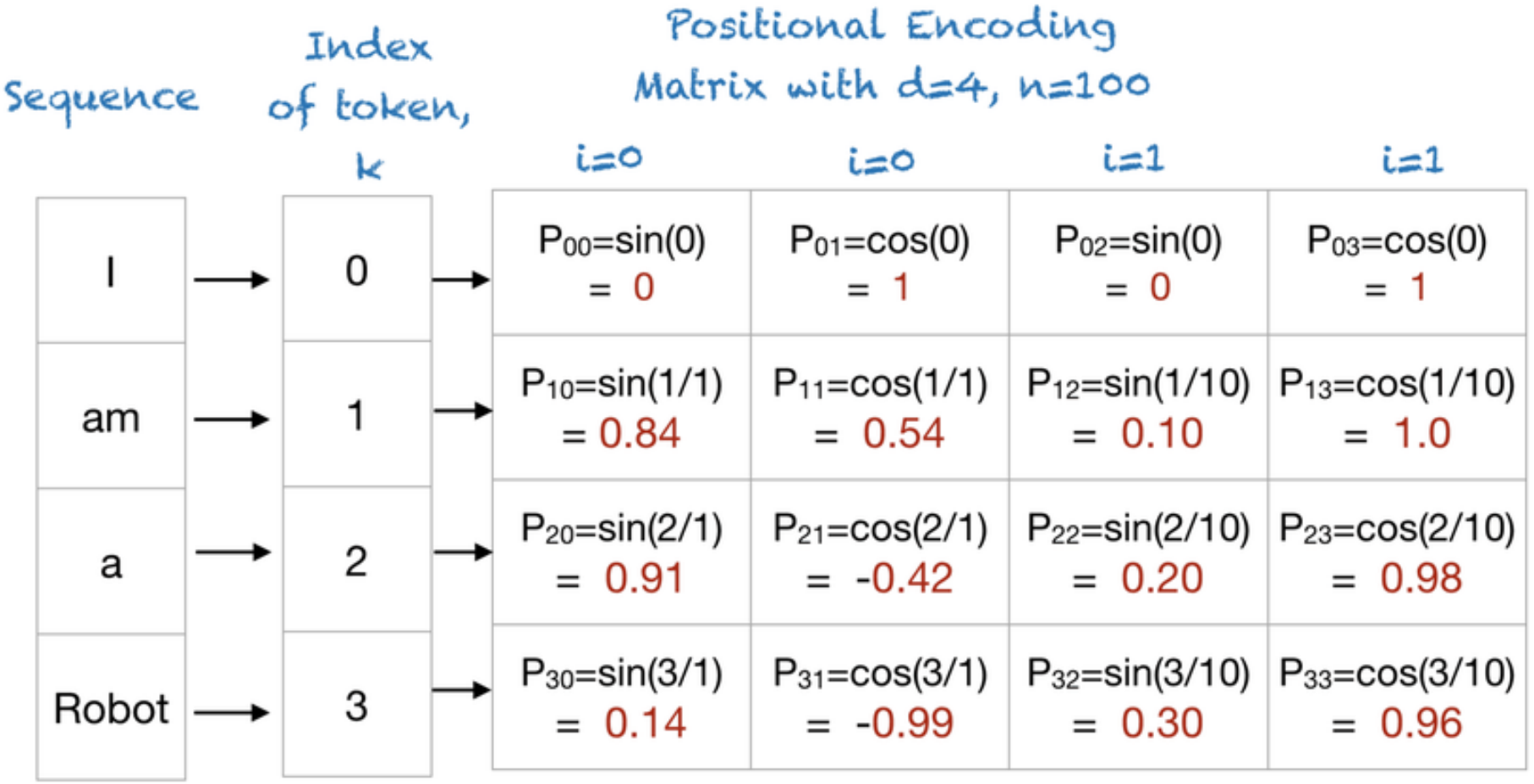
## Positional Embeddings

$$\mathrm{PE}_{(pos,2i)} = \mathrm{sen}\left(\frac{pos}{10000^{\frac{2i}{d_{\mathrm{model}}}}}\right)$$

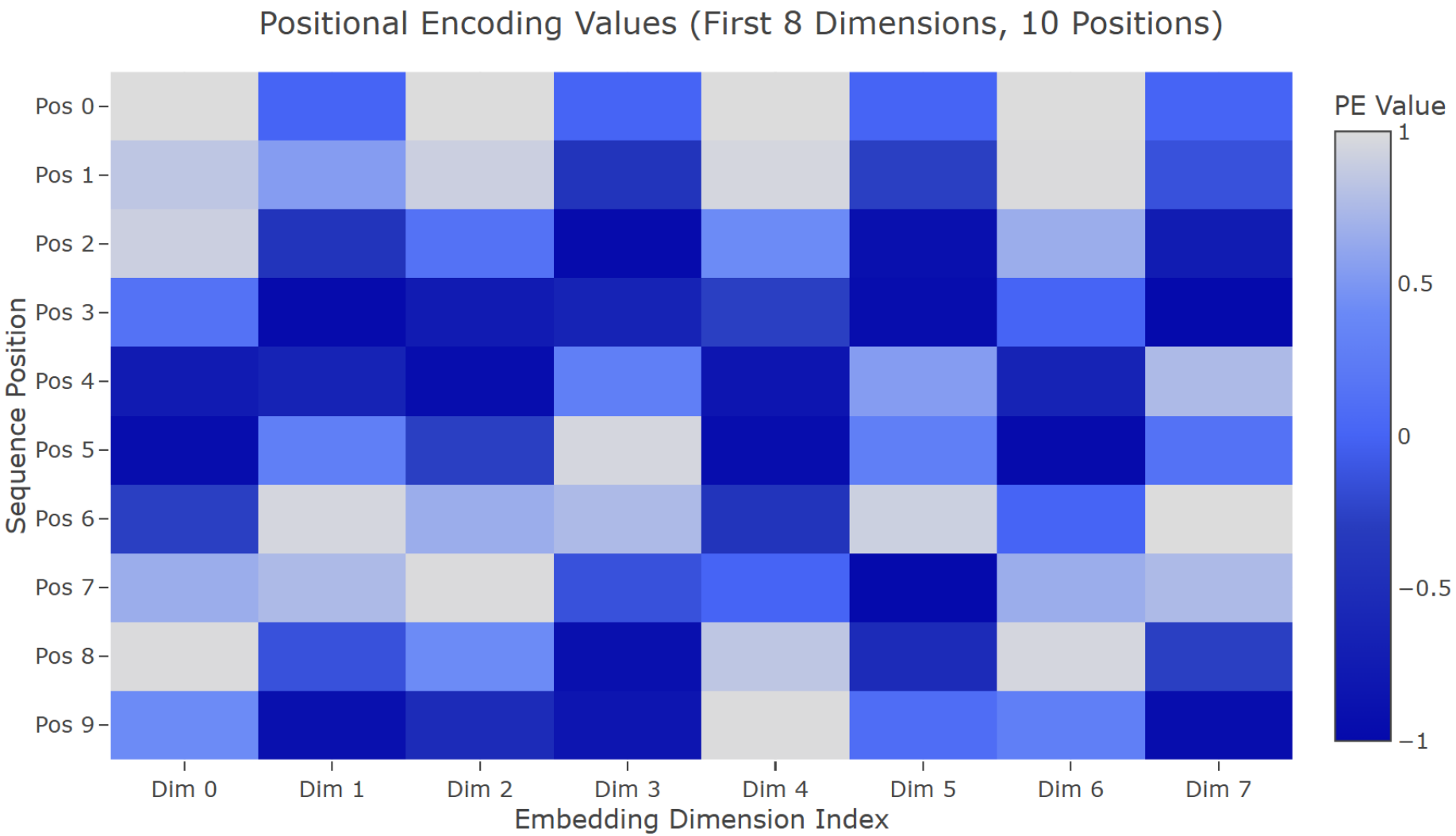
$$\mathrm{PE}_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{\mathrm{model}}}}}\right)$$

- $pos$: a posición do token na secuencia de entrada, comezando en 0.
- $i$: o índice da dimensión no embedding posicional; xera pares de dimensións $(2i, 2i+1)$.
- $d_{\mathrm{model}}$: a dimensión total do vector de embeddings (por exemplo, 512 ou 768).
- $10000^{\frac{2i}{d_{\mathrm{model}}}}$: factor de escala que controla a frecuencia de oscilación en cada dimensión.

# Positional Embeddings



# Positional Embeddings



# Input to Transformers

Pequeno exercicio para resumir a entrada aos Transformers:

- Dada esta secuencia de entrada: "I am I".
- Empregade o tokenizador de BERT de HF para obter os token IDs (ignorando os tokens especiais).
- No seguinte notebook podedes obter os embeddings que xa ten o modelo de BERT adestrado para os vosos tokens: `https://colab.research.google.com/drive/1vSsbxi1_8OVzXsxaqZUstQCQ_kBLlhQo?usp=sharing`.
  Seleccionade só as dúas primeiras posicións do embedding.
- Calculade agora os positional embeddings (asumindo que tamén son de dimensión 2). Non fai falta aplicar o factor de escala para este exercicio; podedes usar simplemente $sin(pos)$ e $cos(pos)$.
- Xerade os embeddings finais. Que observades?

# Input to Transformers

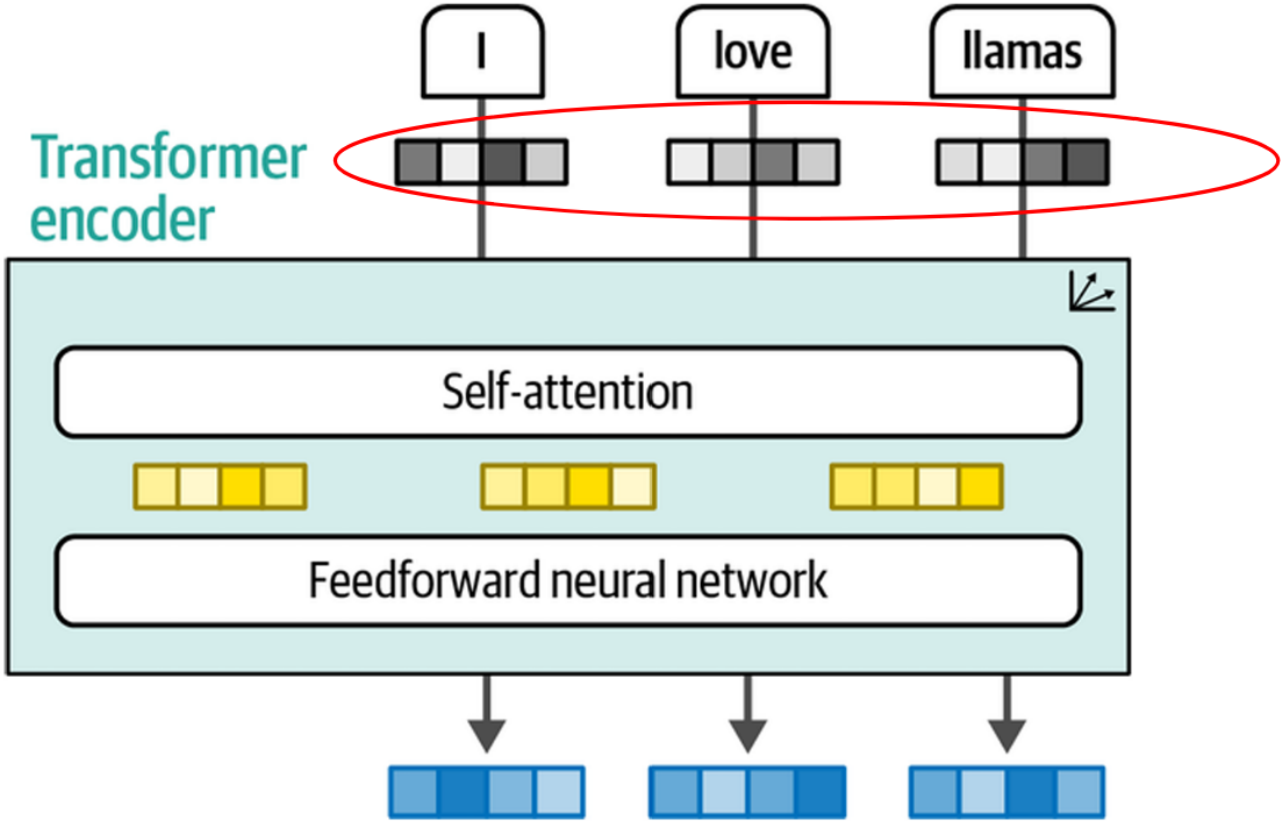



Figura: Figura extraída do libro “Hands on Large Language Models”.

# Encoder Stack

# Encoder Stack

## Encoder Stack

- A función do encoder é coller os $final_{\text{embedding}}$ e xerar embeddings contextuais, tamén chamados **hidden states**.
- O encoder ten dous compoñentes principais: a **self-attention** e unha rede neuronal *feed-forward* (**FFNN**).
- Como veremos, en realidade realízanse varias operacións de **self-attention en paralelo** para captar diferentes relacións contextuais.
- Ademais, o proceso de codificación non sucede nun único paso, senón que en realidade existe un **stack** de capas de **encoders** idénticas.

# Self-Attention

## Mecanismo de Attention

- Sabemos que a atención permite ao modelo centrarse en partes específicas da secuencia de entrada, pero como funciona realmente o mecanismo de atención?
- En primeiro lugar, dérivanse 3 vectores a partir dos *input embeddings* de cada token:
  - **Query (Q)**: o que esta palabra (token, usamos palabra por simplicidade) está "buscando" nas demais.
  - **Key (K)**: o que cada palabra "ofrece", é dicir, como pode ser relevante para as outras.
  - **Value (V)**: a información real que esta palabra achega se se considera relevante.

# Self-Attention

## Mecanismo de Attention

Exemplo: Dada a secuencia de entrada "A rapaza le un libro", estase a calcular a atención para o token "le":

- O *input embedding* do token "le" podería xerar unha **Query** que busque suxeitos.
- O *input embedding* do token "rapaza" xera unha **Key** que a identifica como un posible suxeito.
- Como a **Query** de "le" se aliña ben (score de similaridade) coa **Key** de "rapaza", préstaselle atención.
- O **Value** de "rapaza" incorpórase ao contexto que se xera para "le".

# Self-Attention

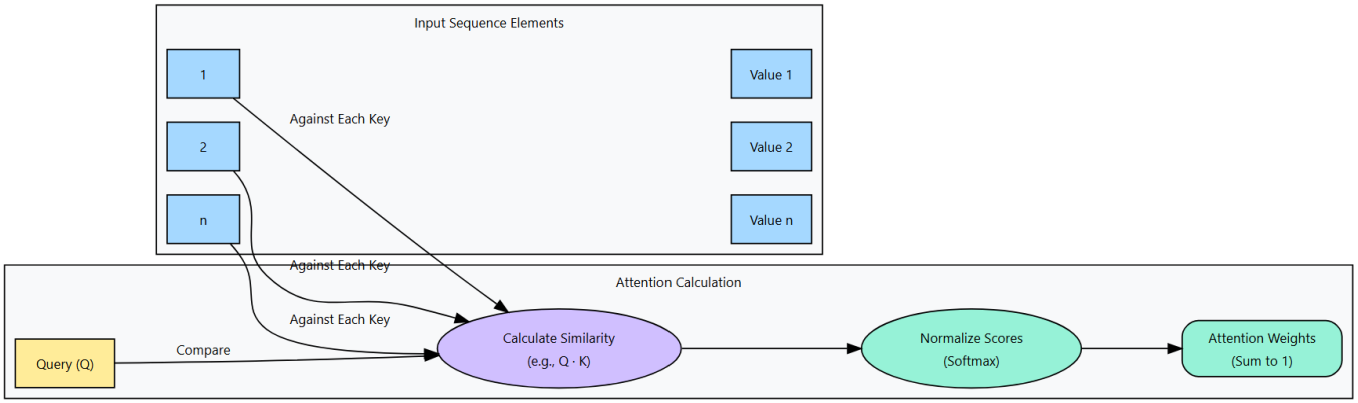



Figura: Diagrama de alto nivel de como funciona a atención.

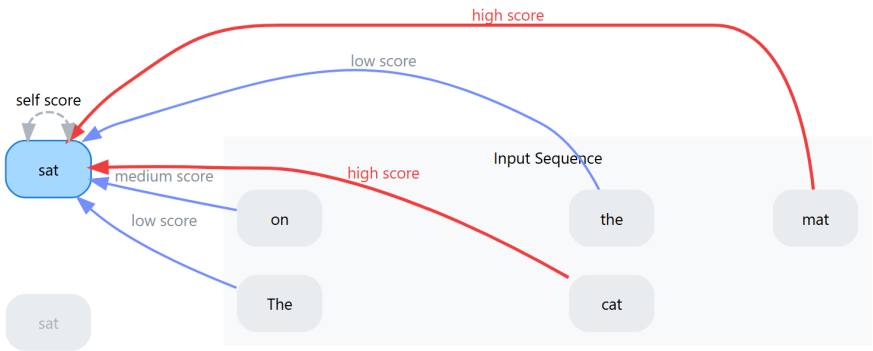
## Pregunta

Pero, como se derivan Q, K y V? Que quere dicir iso de “buscar”, “ofrecer”, etc.?

# Self-Attention

## Self-Attention

- Trátase do primeiro paso dentro do Encoder.
- Baséase no concepto de atención, é dicir, en fixarse nas partes máis relevantes. A diferenza é que aquí, ao construír un embedding para un token, o modelo fíxase na **mesma secuencia** de **entrada** (incluído o **propio token**).
- Por exemplo, dada a entrada "*The animal didn't cross the street because it was too tired*". A capa de *self-attention* serve para aclarar a quen se refire "it": a "animal" ou a "street".



# Self-Attention

## Generando Q, K y V

$$Q = XW^Q$$
$$K = XW^K$$
$$V = XW^V$$

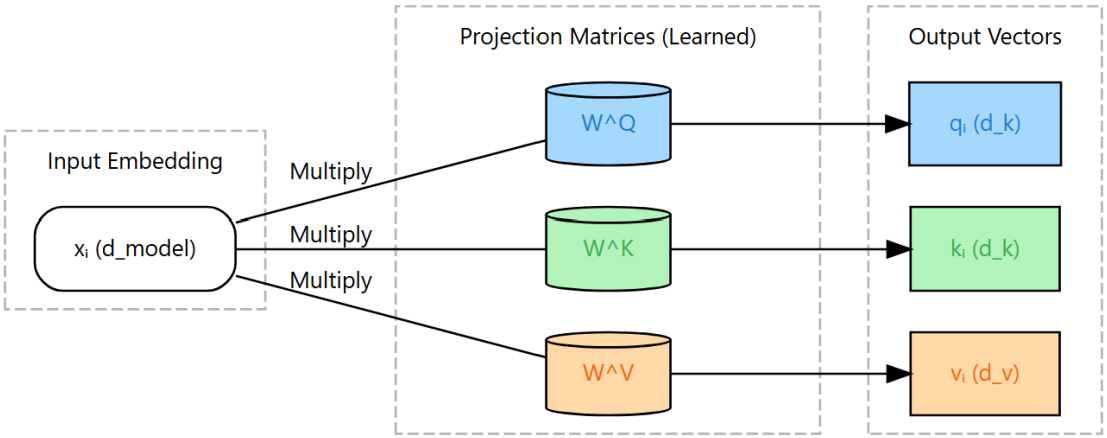
onde:

- $X \in \mathbb{R}^{n \times d}$ é a matriz de embeddings da secuencia (con $n$ tokens e dimensión $d$).
- $W^Q, W^K \in \mathbb{R}^{d \times d_k}$ son matrices de pesos para consultas e claves.
- $W^V \in \mathbb{R}^{d \times d_v}$ é a matriz de pesos para valores.
- $Q, K \in \mathbb{R}^{n \times d_k}$, $V \in \mathbb{R}^{n \times d_v}$ son as matrices resultantes.
- As matrices de Q e K teñen que ter a mesma dimensión ($d_k$) para poder ser comparadas mediante produto escalar.

# Self-Attention

## Generando Q, K y V

Esencialmente estamos a proxectar os *input embeddings* nun espazo vectorial diferente.



# Self-Attention

## Generando Q, K y V

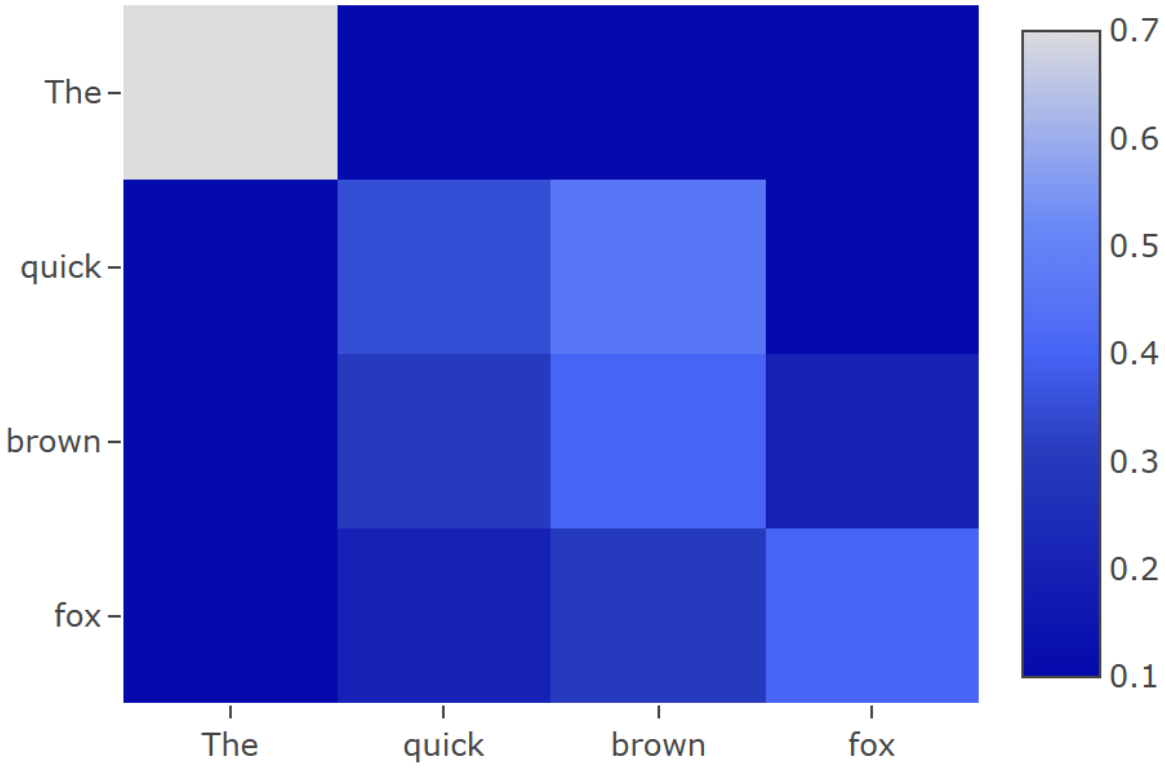
- As matrices W son parámetros adestrables (durante o pre-training ou o fine-tuning, verémolo máis adiante).
- $d_k \neq d$ no caso da multi-head attention. Pero normalmente asúmese que $d_k = d_v$.

# Scaled Dot-Product Attention Mechanism

## Scaled Dot-Product Attention Mechanism

- Esta é a base que **calcula** canta **atención** debe prestarlle cada elemento ao resto da secuencia (incluído el mesmo).
- $\text{Attention}(Q, K, V) = \underbrace{\text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)}_{\text{pesos de atención}} V$
- $QK^\top \in \mathbb{R}^{n \times n}$ (matriz de similitudes entre tokens)
- $\text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)$ (distribúe a atención, normaliza)
- Dividimos entre $\sqrt{d_k}$ porque sen este escalado, se $d_k$ é grande, os produtos escalares poden ser grandes en magnitude, e a softmax pode levar a extremos (valores preto de 0 ou 1), o que faría que o modelo aprenda mal. O escalado estabiliza o gradiente.
- A saída é unha **representación contextualizada** da **secuencia** de tamaño $n \times d_v$.

# Attention Weights




## Exercicio

https://colab.research.google.com/drive/1r0JOvKhIy2TjPaqpcddrhCF7hF_bziAI?usp=sharing

# Exercicio

Pequeno **exercicio** do cálculo de **self-attention a man**:

- Imaxinade que temos a frase "aprendo IA rápido" como entrada do encoder.
- Xa temos as matrices Q, K e V calculadas e, por simplicidade, son iguais:

$$Q = K = V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$$

- Calculade a matriz de scores $S = Q \cdot K^T$.
- En lugar de calcular o softmax (expoñenciais e división), asignade pesos proporcionais para cada fila da matriz de scores, asegurando que sumen 1 e respectando a relación de magnitude. Estes serán os *attention weights*.
- Calculade o novo embedding de cada palabra multiplicando os seus *attention weights* pola matriz $V$.

# Multihead Self-Attention

## Pregunta

- Pensemos na frase, "The quick brown fox jumps over the lazy dog".
- Imaxinade que estamos codificando o embedding do token "jumps", a que deberiamos prestarlle máis atención?

# Multihead Self-Attention

## Pregunta

- Pensemos na frase, "The quick brown fox jumps over the lazy dog".
- Imaxinade que estamos codificando o embedding do token "jumps", a que deberiamos prestarlle máis atención? Ao suxeito ("fox"), ao obxecto ("dog") ou ao atributo ("quick")?
- Este é un claro exemplo no que o cálculo dunha única atención non é suficiente.
- Esta observación conduce ao concepto de **Multi-head Self-Attention** ou múltiples atencións en paralelo.
- A idea: que cada cabeza vexa a secuencia desde un espazo distinto, capturando diferentes relacións entre tokens.

# Multihead Self-Attention

## Multihead Self-Attention

- A cada cálculo chámaselle **cabeza de atención**. Cando utilizamos multi-head attention con $h$ cabezas de atención, $d_k = d/h$, é dicir, cada cabeza ve un "subespazo" diferente da entrada.
- $W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d \times d_k}$ son matrices de pesos adestrables específicas para cada cabeza $i$.
- $Q, K, V \in \mathbb{R}^{n \times d}$ proveñen da mesma secuencia de entrada. Cada cabeza toma a entrada e proxéctaa coas súas matrices adestrables:

$$Q_i = QW_i^Q \in \mathbb{R}^{n \times d_k}, \quad K_i = KW_i^K \in \mathbb{R}^{n \times d_k}, \quad V_i = VW_i^V \in \mathbb{R}^{n \times d_k}.$$

- $Q_i, K_i, V_i \in \mathbb{R}^{n \times d_k}$ son as versións proxectadas utilizadas pola cabeza $i$.

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i)$$

# Multihead Self-Attention

## Multihead Self-Attention

- Ao final do proceso de atención, concaténase a saída das diferentes cabezas de atención para obter un vector de dimensión $d$:
- $\text{Concat}(\text{head}_1, \dots, \text{head}_h) \in \mathbb{R}^{n \times d}$, onde cada $\text{head}_i \in \mathbb{R}^{n \times d_k}$ é a saída da cabeza $i$.
- Esta saída concatenada pásase por unha capa de proxección lineal final para mesturar a información aprendida polas diferentes cabezas:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

## Mellora de paralelización respecto ás biLSTM

As cabezas de atención son independentes. Isto facilita a paralelización en GPU/TPU, distribuíndo as cabezas como batches separados e acelerando o cálculo sen alterar a arquitectura.

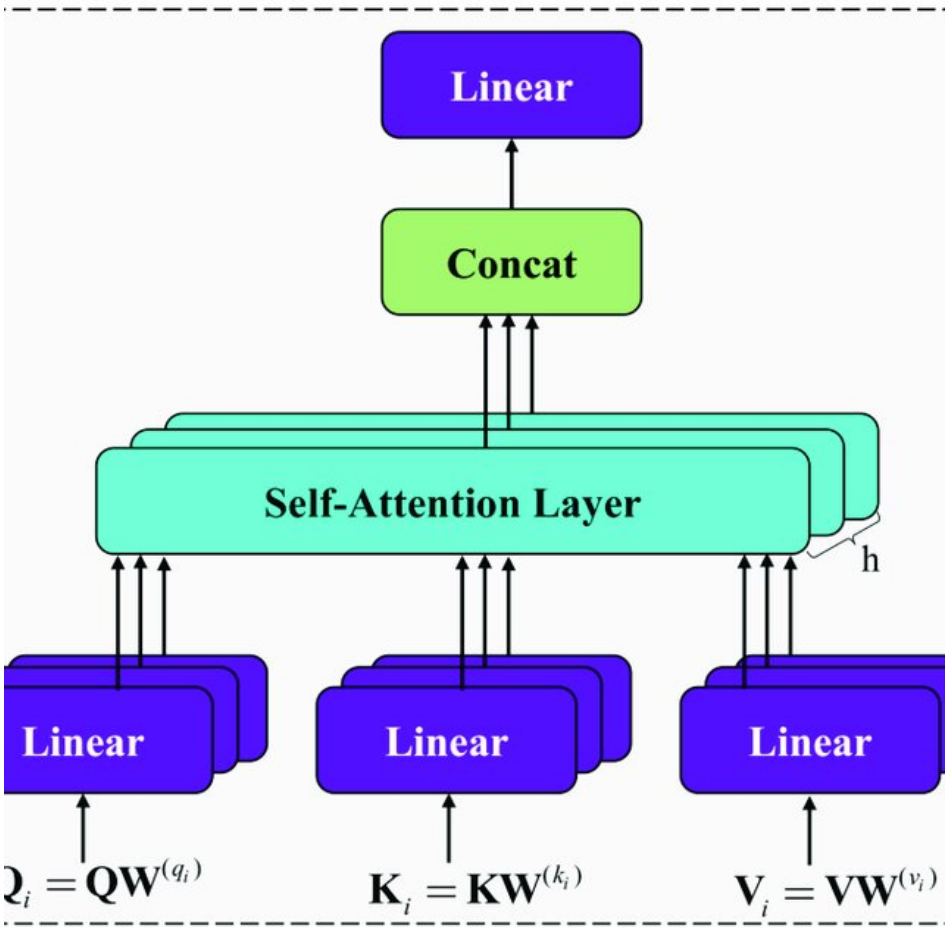
# Multihead Self-Attention

## Ejercicio

Volvemos ao Colab, que pasa se cambiamos head?

`https://colab.research.google.com/drive/1r0JOvKhIy2TjPaqpcddrhCF7hF_bziAI?usp=sharing`

# Multihead Self-Attention



# Encoder Stack

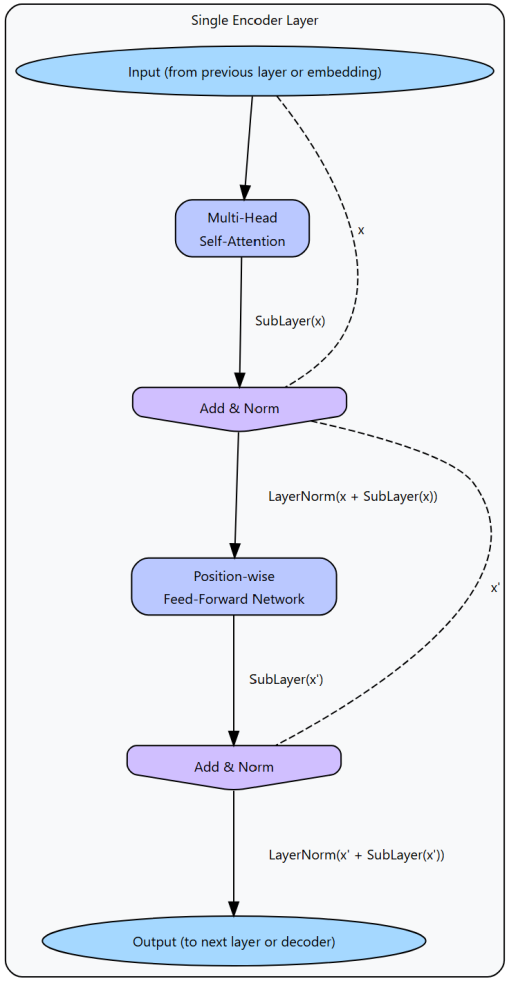



Figura: Esta Figura é só unha capa, pero podería haber N capas apiladas do encoder.

# Feed-Forward Neural Network (FFNN)

## FFNN

- A FFNN aplícase de forma independente a cada token:

$$\text{FFN}(x) = \text{máx}(0, xW_1 + b_1)W_2 + b_2$$

- $W_1, W_2$ y $b_1, b_2$ son parámetros entrenables.
- Aplica non linealidade con ReLU máx$(0, .)$.

# Add & Norm

## Add & Norm

- Engádese unha conexión residual para facilitar o aprendizaxe:

$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

- Onde $x$ é a entrada ao submódulo e Sublayer($x$) é a súa saída. O submódulo pode ser a multi-head attention ou a FFNN.
- A idea clave é sumar a entrada orixinal á saída dun submódulo (por exemplo, a saída do bloque de atención ou da FFNN).
- Isto crea un atallo ou "skip connection" que permite que a información flúa directamente cara a capas superiores, sen degradarse.
- Axuda a estabilizar o adestramento e evitar o desvanecemento do gradiente.

# Saída do Encoder

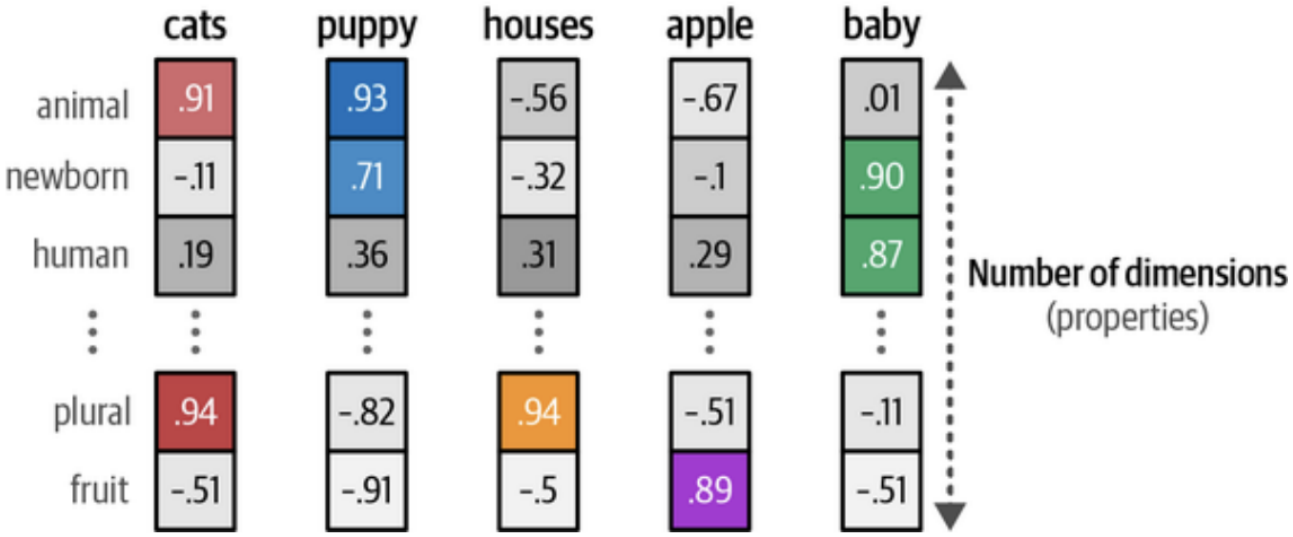



Figura: Á saída do encoder, teremos unha representación en embeddings contextualizados do texto de entrada. Cada dimensión do embedding representa dalgún xeito un concepto (verémolo no seguinte tema cos conceptos de explicabilidade). Figura extraída do libro "Hands on Large Language Models".

# Decoder Stack

# Decoder Stack

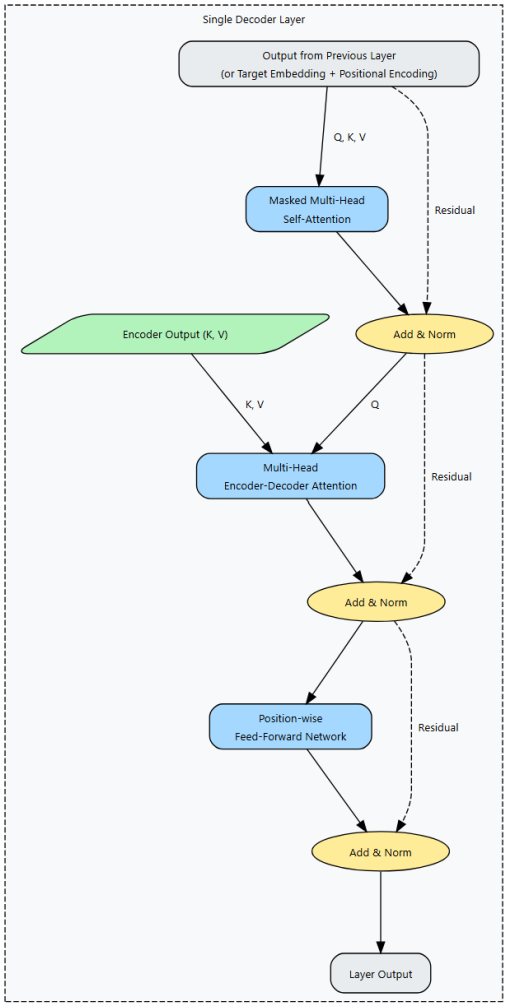



Figura: Esta Figura es solo una capa, pero podría haber N capas apiladas del decoder.

# Decoder Stack

## Decoder Stack

- Debemos lembrar que o **obxectivo** do decoder é xerar a secuencia de saída **elemento a elemento**.
- A diferenza do encoder, que pode procesar toda a secuencia de entrada simultaneamente, o decoder debe predicir o seguinte token baseándose unicamente nos tokens xerados ata ese momento. É o que denominamos un modelo **autorregresivo**.
- Durante o adestramento, non pode anticiparse aos tokens futuros da secuencia de saída que está a construír nese momento, xa que en tempo de inferencia tampouco os vai ter.
- Para adestrar utilízase o **teacher forcing**, no que o decodificador recibe a secuencia de destino correcta (**desprazada cara á dereita**, a miúdo cun token de inicio de secuencia) como entrada para predicir o seguinte token en cada posición.

# Decoder Stack

## Decoder Stack

Exemplo de entrada desprazada:

- "O gato dorme." $\rightarrow$ "<s> O gato" (adoita engadirse un token de inicio de secuencia).
- **Teacher forcing**: durante o adestramento, "obrígase" ao modelo a utilizar as respostas correctas (*ground truth*) das predicións anteriores, en lugar das súas propias predicións previas, que poden ser erróneas e contaminar o aprendizaxe.
- Isto reduce o tempo de converxencia.

# Masked Self-Attention

## Importante

A principal **diferenza** coa *self-attention* do encoder é que aquí, cando se xera o *embedding* contextualizado dun token, só pode prestar atención aos tokens anteriores, xa que serán os únicos dispoñibles en inferencia.

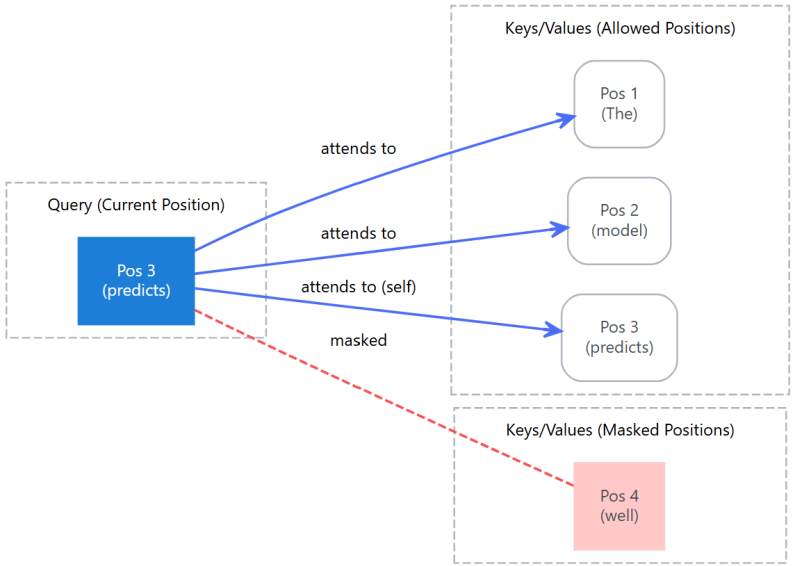
## Masked Self-Attention

- Para iso, aplícase unha **máscara** e a atención calcúlase como:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V$$

- A máscara $M$ é unha matriz que contén: ceros para as posicións permitidas e un valor negativo moi grande $(-\infty)$ para as posicións non permitidas.
- Forza á softmax a asignar probabilidade cero aos tokens non permitidos.

# Masked Multi-Head Self-Attention




## Masked Multi-Head Self-Attention

O proceso de xeneralización a **multi-head** é **análogo** ao do encoder.

# Encoder-Decoder Attention Mechanism

## Encoder-Decoder Attention Mechanism

- Permite ao decoder acceder á representación contextual da secuencia de entrada.
- Calcúlase como:

$$\text{Attention}(Q_{\text{dec}}, K_{\text{enc}}, V_{\text{enc}})$$

- Onde:
  - $Q_{\text{dec}}$: saída do decoder (ata ese punto).
  - $K_{\text{enc}}, V_{\text{enc}}$: saída do encoder.
- Non se aplica máscara: o decoder pode acceder a toda a entrada. Isto débese a que no paso anterior se contextualiza cada token da saída usando só o pasado (grazas á máscara). Despois conéctase coa entrada completa para producir unha **saída informada**.
- Xeneralízase a **multi-head** igual que nos pasos anteriores.

# Encoder-Decoder Attention Mechanism

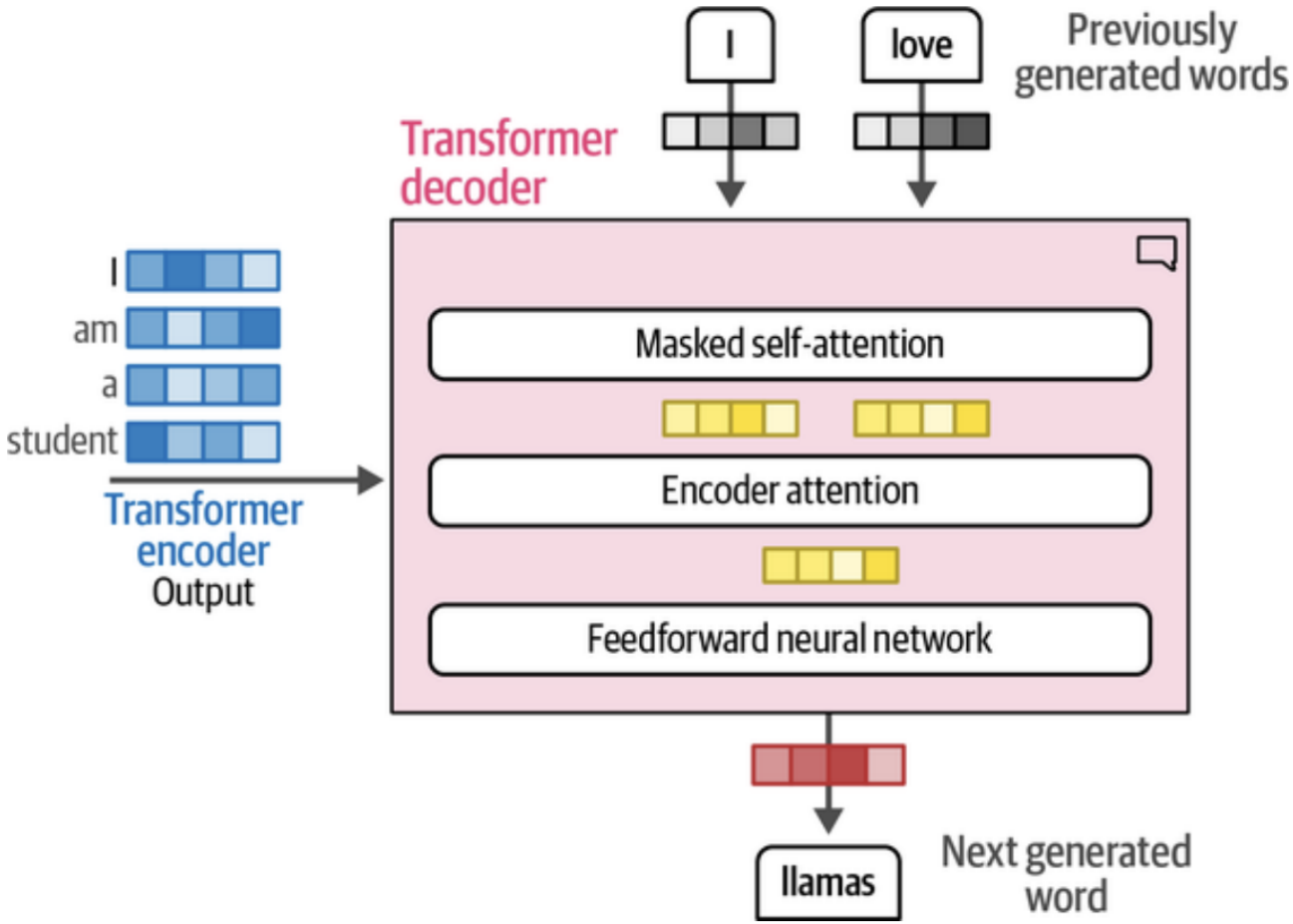



Figura: Figura extraída do libro “Hands on large language models”.

# Final Layers

## Add&Norm e FFNN

Á saída de cada submódulo aplícase unha operación de Add&Norm. O penúltimo paso volve consistir nunha FFNN que aplica non linearidade mediante unha función de activación ReLU.

## Output layer (capa de salida)

- O obxectivo do decoder é xerar texto. Existe unha capa final (*fully connected*) que proxecta a saída ao tamaño do vocabulario:

$$\text{logits} = x_{\text{out}} W^{\top} + b$$

  - $x_{\text{out}} \in \mathbb{R}^{n \times d}$: saída do decoder.
  - $W_E \in \mathbb{R}^{|V| \times d}$ e $b \in \mathbb{R}^{|V|}$ (termo de sesgo opcional), que son parámetros adestrables.

- Aplícase unha **softmax** para obter as probabilidades.

# Modelos seq2seq

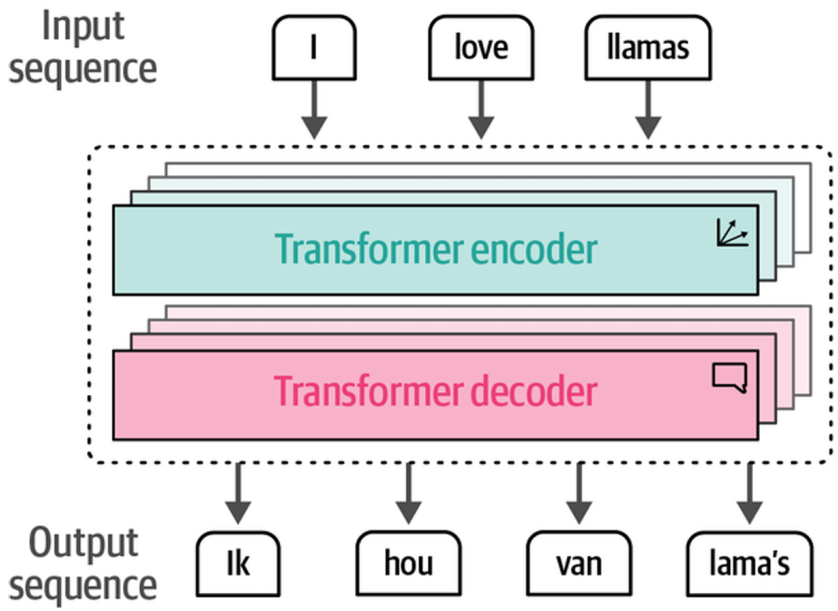



Figura: Fíxadevos en que o decoder tamén ten varias capas apiladas.

**seq2seq**

Os Transformers que constan tanto de Encoder como de Decoder denomínanse modelos **seq2seq**, como, por exemplo, T5, BART, etc.

# Modelos encoder-only

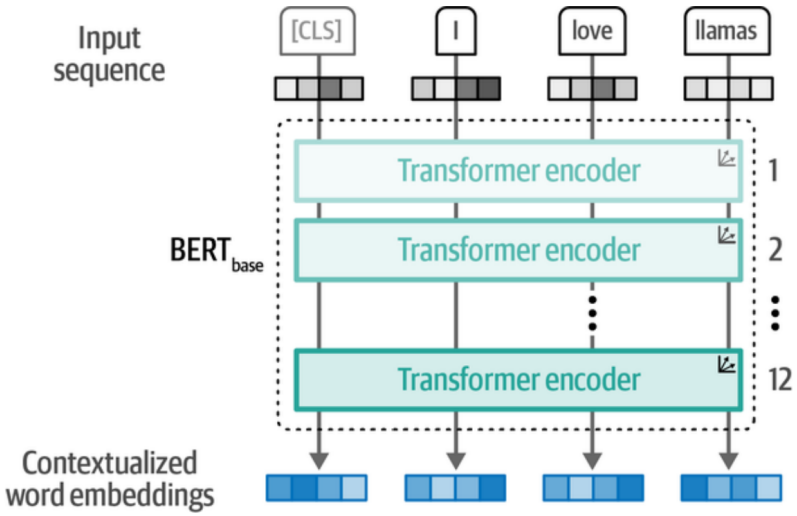
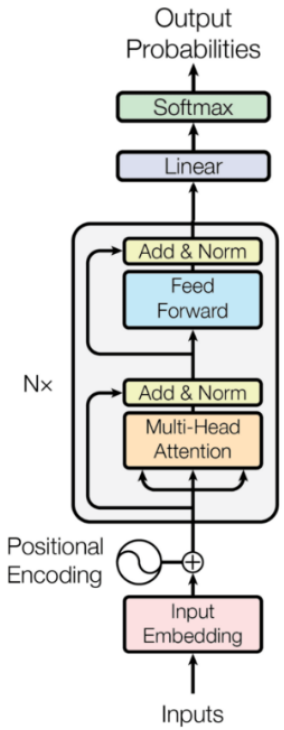



Figura: Figura extraída do libro "Hands on Large Language Models".

## Encoder-only

Estes embeddings contextualizados poden empregarse como entrada para un modelo tradicional ou, como veremos máis adiante, pódese engadir unha *classification head*.

# Modelo de BERT




| | BERT Base | BERT Large |
|---|---|---|
| **Number of embedding dimensions, d_model** | 768 | 1024 |
| **Number of encoder blocks, N:** | 12 | 24 |
| **Number of attention heads per encoder block:** | 12 | 16 |
| **Size of hidden layer in feedforward network:** | 3072 | 4096 |
| **Total parameters:** | 110 million | 340 million |

# Modelos decoder-only

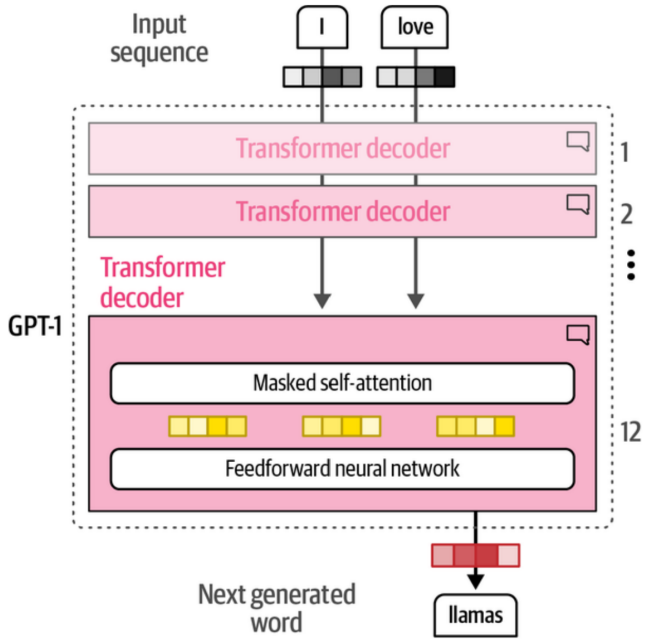



Figura: Figura extraída do libro "Hands on Large Language Models".

## Ollo

Non existe un mecanismo de *encoder-decoder attention*, xa que están deseñados para seren unicamente modelos autorregresivos a partir da entrada (*prompt*).

# Modelos decoder-only

## Modelos xenerativos de linguaxe

Nesta categoría entran o que hoxe en día coñecemos como LLMs.
Os LLMs son modelos xerativos da linguaxe que empregan un decoder para ir ampliando o *prompt* de maneira autorregresiva cos tokens que van xerando.

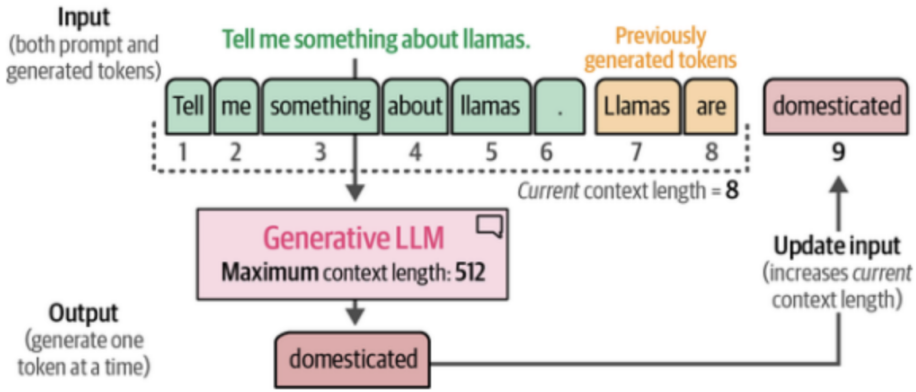



Figura: Figura extraída do libro "Hands on Large Language Models".

# Atención densa vs Atención dispersa

## Atención densa vs Atención dispersa

- Atención densa:
  - Cada token da secuencia presta atención a todos os demais tokens.
  - Se tes unha secuencia de lonxitude $n$, xérase unha matriz de atención de tamaño $n \times n$.
  - A complexidade algorítmica en tempo e memoria é cadrática: $O(n^2)$.
- Atención dispersa:
  - En lugar de prestar atención a todos os tokens, selecciónanse aqueles aos que se lles presta atención, é dicir, os máis relevantes.
  - Emprega un mecanismo dinámico de selección das conexións relevantes.
  - Complexidade típica: $O(n \cdot k)$ con $k \ll n$ (cada token presta atención só a $k$ veciños relevantes).
  - Por exemplo, o modelo LongFormer.

# Multi-Query Attention

## Multi-Query Attention (MQA)

- Xa vimos a Multi-Head Attention (MHA). Nela, $W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d \times d_k}$ son matrices de pesos específicas para cada cabeza.
- É dicir, cada cabeza ten o seu propio espazo de atención.
- En MQA, cada cabeza ten a súa propia matriz $W_i^Q$, pero $W^K$ e $W^V$ son compartidas entre todas as cabezas.

$$K_{\text{shared}} = KW_{\text{shared}}^K \in \mathbb{R}^{n \times d_k}, \quad V_{\text{shared}} = VW_{\text{shared}}^V \in \mathbb{R}^{n \times d_k}$$

- MQA permite aforrar **tempo** e **memoria**.

# Exercicio

Considerade un Transformer con $L$ capas de atención (cunha única cabeza por capa), onde $L$ é constante. Cada capa de atención permite que a representación de calquera token incorpore información de todos os demais tokens da secuencia. A predición da seguinte palabra obtense a partir da representación do último token.

1. Cal é a lonxitude do camiño computacional entre o símbolo $y_1$ e a predición $p_{LM}(y_T \mid y_{<T})$. Comparádeo co dun modelo RNN e indicade a orde de crecemento con respecto á lonxitude da secuencia $T$.

# Repaso e seguintes pasos

## Repaso e seguintes pasos

Neste tema vimos:

- Que é o mecanismo de atención.
- A implementación detallada da arquitectura Transformer:
  - Desde a entrada ao modelo: tokenización, *embedding lookup table* e *positional encodings*.
  - A implementación das *encoder stacks* e *decoder stacks*.

Nos seguintes temas profundaremos en:

- Como aprenden estes modelos. Concepto de preadestramento e os seus tipos.
- *Embeddings* a nivel de documento e de oración.
- Modelos preadestrados: plataformas como Hugging Face ou Ollama.
- Introdución á explicabilidade.